Join the conversation

Back to ;login: Online

;login:

Package Hallucinations: How LLMs Can Invent Vulnerabilities

A Comprehensive Study that Sheds Light on a Systemic and Persistent Threat

June 17, 2025

Research

Authors:

Joseph Spracklen, Raveen Wijewickrama, A.H.M. Nazmus Sakib, Anindya Maiti, Bimal Viswanath, Murtuza Jadliwala

Article shepherded by:

Rik Farrow

Imagine setting your car's navigation system to guide you home after a long, exhausting day. You trust it implicitly; it has reliably navigated you through countless commutes. But this evening, instead of your familiar driveway, the AI confidently directs you to an empty field, miles from anywhere recognizable. At first, it might seem a harmless glitch that only inconveniences your time. But what if someone anticipated this mistake, lying in wait precisely because they knew the AI occasionally misled

drivers into this isolated spot? Suddenly, a minor technological hiccup transforms into a serious threat.

This unsettling scenario mirrors a new class of vulnerability that has been discovered in the powerful AI systems increasingly integrated into our daily lives. While our navigation apps remain stable and trustworthy, Large Language Models (LLMs) have swiftly begun reshaping critical areas such as software development, often without the same stability or reliability. Like our hypothetical navigation error, these language models can confidently and convincingly deliver incorrect or even completely fabricated information, a phenomenon referred to as hallucinations. In software development, one way this can manifest are "package hallucinations," which occur when an AI coding assistant recommends a third-party software package that simply does not exist. As in the navigation analogy above, this may seem harmless at first, but this particular type of hallucination opens a new attack vector for malicious actors that has implications for the software supply chain and everyday users alike.

Our research aims to provide the first comprehensive analysis of package hallucinations across a variety of models, settings, and programming language. We not only quantify how often this phenomenon occurs but examine other important characteristics such as persistence, self-detection, and mitigation strategies.

This article serves as an abbreviated version of our **research paper** [7], which is set to appear at the USENIX Security Symposium 2025, with expanded discussion on navigating the present environment of highly influential, but imperfect, generative AI systems.

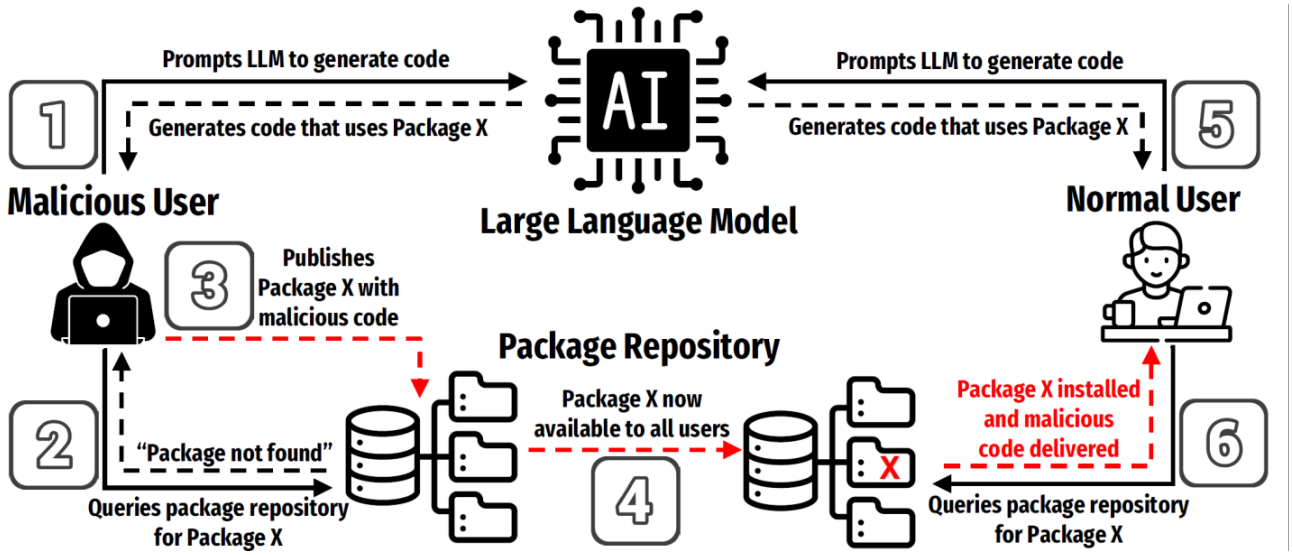

Figure 1: Flowchart for Exploiting Package Hallucinations. An attacker prompts an LLM for code (1) and the generated code contains a hallucinated package name (2). The attacker publishes a package containing malicious code using the hallucinated name (3) which is now available to any user (4). Now, the next time a normal user asks a similar question to the LLM (5) and the generated code contains the same hallucinated package name, that user is at risk of installing the malware on their device (6).

Anatomy of a Package Hallucination Attack

The emergence of package hallucinations as a viable threat stems from the interaction of three distinct elements: open-source package repositories, proficient coding models, and hallucinations in LLMs.

Open-source repositories

Open-source software repositories like PyPI for Python and npm for JavaScript play a crucial role in modern software development. They host millions of packages, providing reusable code components that drastically accelerate software development. Their accessibility and ease of use allow developers to rapidly integrate third-party libraries at no cost. While the centralization and ease of use has been convenient for users, their open-source nature means uploads are anonymous and without rigorous oversight. As such, attackers have long recognized these repositories as fertile ground for planting malware [1]. Package confusion attacks, where attackers deliberately publish malicious packages with names deceptively

similar to legitimate ones, have plagued these repositories for years [2]. Package hallucinations represent the latest evolution of this threat, amplifying traditional package confusion attacks with AI-driven recommendations, creating a new vector for supply-chain compromises.

Coding Models

The rise of competent and proficient coding models including GitHub Copilot, Cursor, ChatGPT, Claude, and various open-source alternatives has transformed software development [3]. These models are already responsible for generating a significant portion of production code, a trend poised to increase rapidly as the technology improves [4]. While these systems offer incredible speed and convenience, they are far from infallible. AI-generated code is not immune to mistakes and can include syntax errors, hidden vulnerabilities, or outright fabrications [5]. Despite these issues, the proficiency of these coding models create incredible efficiency gains that add value to even the most skilled developers. The immense pressure for rapid software delivery means developers often have limited time or resources to thoroughly vet AI-produced outputs. This reliance on AI-generated code is creating a growing opportunity for inadvertent acceptance and implementation of flawed or malicious code.

Hallucinations

Hallucinations are a natural by-product of how LLMs operate. These models generate text probabilistically, meaning their outputs are inherently non-deterministic and subject to random sampling errors. While this randomness enables creativity and engaging content, highly desirable qualities, it also introduces the possibility of generating entirely fabricated or factually incorrect information [6]. Examples of hallucinations include models citing non-existent research papers, inventing plausible-sounding historical events, or confidently providing incorrect technical instructions. Such hallucinations occur frequently enough that developers and users must maintain vigilance and scrutinize output. Advances such as Retrieval-

Augmented Generation (RAG) and agentic models have significantly reduced the frequency and severity of these issues, but have not entirely eliminated them. Hallucinations are truly both a feature and a bug, and striking the right balance between accuracy and creativity is a major decision point when introducing models to the public.

Package Hallucinations

Package hallucinations occur when an LLM confidently recommends software packages that do not actually exist in the default package repositories. While seemingly harmless, merely imaginary names generated by an AI, these fictitious packages pose severe security risks. Imagine an attacker that notices repeated recommendations for a non-existent package name. They swiftly publish a malicious version of this phantom package on a public repository. Developers following AI recommendations then inadvertently download and install malware disguised as the seemingly legitimate package. The attack is deceptively simple and inexpensive to execute, akin to phishing attacks: low-risk, high-reward scenarios where minimal effort or financial investment can yield substantial damage. Figure 1 illustrates a hypothetical attack flow, showcasing how an initially benign hallucination can rapidly escalate into a dangerous security compromise.

Experiment Design

The goal of our experiment was to conduct a rigorous and comprehensive test to conclusively quantify how often these package hallucinations occur and how they are impacted by different models, languages, and settings. In short, we chose a variety of models and used each model to generate a large number of code samples which we would then test for the presence of package hallucinations. Detecting package hallucinations given only a sample of code is a very difficult task that would be extremely error prone at best, so instead we used the code samples themselves as a way to

prompt the model for more information, similar to what we envision a normal user might do as part of their workflow. Imagine trying to run commands you received from an LLM and receiving an error saying that the package or module was not found. We felt the following 3 scenarios were plausible and formed the basis for our hallucination detection:

1. Examine the full model response for directions to install a package with *"pip install"* or *"npm install"*. Models sometimes include these instructions in the narrative of their response, outside of the actual code block. A package hallucination of this type serves as the most dangerous scenario, because the model is directly recommending that the user run code that would immediately result in malware being downloaded and installed.
2. Prompt the model for a list of packages needed to run the code sample.
3. Ask the model to recommend a list of packages that could help answer the original question.

Collecting the results of the three different tests, we would then simply compare each given list to the master list of packages obtained from the relevant repository. If a given package was not found then it was counted as a package hallucination and our overall measurement was hallucination rate, which is simply a ratio of hallucinated packages generated by the model over the number of total packages generated. There were several other important aspects of the experiment that are relevant for the remaining discussion:

- **Dataset:** We wanted to prompt the models with a very broad range of coding questions, covering as many conceivable topics as possible to test not only the most popular questions but also more niche and obscure subjects. It was also important to include realistic questions that an actual user could plausibly ask. To accomplish both of these goals we used two distinct datasets: an LLM generated set of prompts constructed from scraped repository description data and a set of popular questions scraped from StackOverflow relevant to the

programming languages in question. In total, each dataset including about 9,500 prompts, for a total of ~19,000 prompts that were given to each model.

- **Models:** The goal was to test a representative sample of the best models available from both open-source and commercial options. We used the HumanEval leaderboard to filter the best performing models at the time our research started, which you can see in Figure 3. Note that this project began in February of 2024 and was first published in June 2024, so while the models are already dated in according to the lightning fast AI-timeline, these were the top performing models when research was started.
- **Languages:** We conduct all tests using two programming languages: Python and JavaScript. These two languages are extremely popular and also represent the two largest open-source package repositories: PyPI and npm. These are also the primary languages used in testing the LLM-generated code quality benchmarks, making them a natural choice.

Model	Parameters	License	Open Source
ChatGPT 4.0	Unknown	Commercial	✗
ChatGPT 4.0 Turbo	Unknown	Commercial	✗
ChatGPT 3.5 Turbo	Unknown	Commercial	✗
CodeLlama	7B, 13B, 34B	Free	✗
DeepSeek	1.3B, 6.7B, 33B	Free	✓
Magicoder	6.7B	Free	✓
WizardCoder	34B	Free	✓
Mistral	7B	Free	✓
Mixtral	8x7B	Free	✓
OpenChat	7B	Free	✓
WizardCoder-Python	7B	Free	✓
CodeLlama-Python	33B	Free	✓

Figure 2: Models used for testing

Key Findings

Prevalence

Our study identified that all 16 tested coding models exhibited notable rates of package hallucinations, averaging 19.6% across the board. Notably, commercial models performed significantly better, with hallucination rates around 5%, compared to open-source models averaging around 21%. These figures underline that package hallucination is a systemic issue affecting even state-of-the-art commercial offerings, not merely a quirk of small or less capable models. This ubiquity underscores a significant concern: as coding assistance from AI becomes commonplace, the likelihood of unintentionally propagating these hallucinations through software ecosystems grows. The discrepancy between commercial and open-source models also highlights the necessity for ongoing vigilance and improvement, especially within the open-source community, which many developers rely on due to its accessibility, transparency, and adaptability (and of course, cost).

We were also struck by the breadth of fictitious packages confidently recommended by the tested models. In total, **205,474** unique non-existent packages were generated during testing. Each unique package name represents a potential malicious package that could be uploaded to an open-source repository, ready to be downloaded by unsuspecting developers. This vast number indicates not only the scope of the problem but also the ease with which attackers could find multiple opportunities to introduce harmful code. With so many plausible-sounding yet fictitious names, the odds of successfully tricking developers are substantially heightened, posing a very significant risk for software supply-chain security.

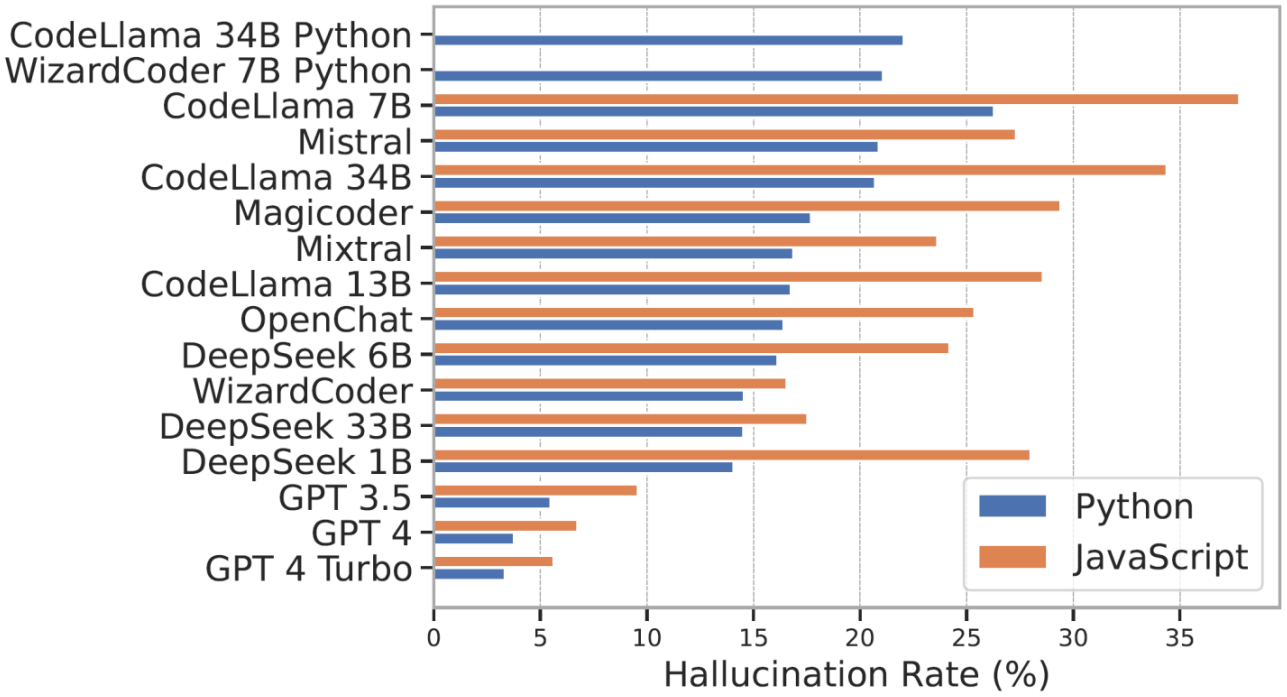

Figure 3: Package Hallucination Rate for Each Model Tested

Persistence

An important aspect of package hallucinations as a threat vector is their persistence, how frequently a hallucinated package reappears in model outputs. If hallucinations were sporadic, single-occurrence errors, their potential threat would be limited. However, our findings demonstrated significant repetition: approximately 45% of hallucinated packages were consistently regenerated every time a model was queried with the same prompt, and about 60% reappeared at least once in 10 subsequent prompts. This recurrence substantially increases their threat profile, providing attackers with reliable and predictable targets for malicious exploitation. The predictability of these hallucinations is something that malicious actors can leverage to their advantage. Essentially, persistent hallucinations create a sustained opportunity for attacks, as attackers can anticipate specific phantom packages appearing again, ensuring their maliciously crafted packages find an audience.

Self-Detection

An interesting, and unexpected, discovery was the ability of certain models, particularly ChatGPT and DeepSeek, to self-detect their hallucinations effectively. When these models generated hallucinated packages, they subsequently exhibited an 80% accuracy rate in identifying these packages as fictitious upon re-examination. This reveals a promising avenue for real-time self-correction mechanisms, potentially reducing the dissemination of these phantom packages before they propagate further into software development processes. Nevertheless, the initial confident recommendations followed by self-recognition indicate inherent inconsistencies within model logic and point to complexities in the model's internal evaluation processes. Enhancing such self-assessment capabilities could form an integral part of defensive strategies, serving as a vital checkpoint in automated coding environments where manual verification is often impractical.

Mitigation

To address package hallucinations, our experiments explored various mitigation techniques, including Retrieval-Augmented Generation (RAG), fine-tuning, self-correction mechanisms, and a combination of all these methods into an ensemble approach. Remarkably, fine-tuning, in which a model's weights are finely calibrating to increase performance on a specific task, on a hallucination-free dataset dramatically reduced the hallucination rate by over 80%, even surpassing the reliability of commercial models like ChatGPT. However, this came at a significant trade-off: a notable decline in the generated code's overall quality. This finding demonstrates that while current mitigation strategies show considerable promise, they remain imperfect solutions. The performance trade-off observed highlights an ongoing challenge to develop methods that significantly curtail hallucinations without sacrificing critical functionality. Future research must strive toward techniques balancing hallucination reduction and maintaining, or even enhancing, code reliability and quality.

Discussion, Considerations, and Navigating the New Normal

AI Firmly Entrenched in Everyday Life

The widespread occurrence of package hallucinations underscores a broader reality: Large Language Models are no longer experimental novelties and have woven themselves into the fabric of daily life. Autocomplete suggestions finish our emails, customer-service bots triage our questions, and real-time translators bridge language gaps on video calls. HR platforms rely on LLMs to screen résumés, banking apps embed chat interfaces that explain loan terms, smart-home hubs summarize news, control appliances, and adjust thermostats. In classrooms, AI tutors adapt lessons in real time, while in hospitals models draft chart notes and help route prior-authorization requests. Most relevant to this discussion, coding assistants now sit directly inside IDEs, Git workflows, and CI/CD pipelines to generate boilerplate code, recommend libraries, and even write entire modules. As the boundary between human and AI-generated output blurs, our dependence on these systems deepens, magnifying both their benefit and their risks.

Hallucinations Are Here to Stay

For all of the productivity, efficiency, and useful ways LLMs have improved our lives, hallucinations are the tax we pay. Given the probabilistic, non-deterministic nature of LLMs, hallucinations are not simply a bug to be patched but a structural consequence of how these systems operate. This randomness fosters creativity, diversity, and open-ended reasoning that is essential for everything from storytelling to problem-solving. However, it also means that these models will sometimes generate information that sounds plausible but is entirely fabricated.

Great strides in mitigating hallucinations have already been made but

eradicating this phenomenon remains an open challenge, especially in high-stakes or long-tail edge cases. Methods like chain-of-thought reasoning, agentic systems, and Retrieval-Augmented Generation (RAG) only shrink the error surface; they cannot eliminate it. Even if the package hallucination rate was only a fraction of a percent, at the scale of tens of millions of code completions per day that is still tens of thousands of fictitious packages being generated. These mitigation methods will continue to improve, and new methods will likely be developed that continue to diminish the presence of hallucinations. The fact remains, however, that hallucinations are a natural occurrence that directly stems from the basic architecture of an LLM, and it is difficult to envision completely eliminating them without some fundamental change to the entire language model structure.

The Illusion of Verification

A particularly dangerous misconception is that whitelisting package names or using RAG-based validation can fully prevent hallucination-based attacks. This is a common misconception that fools many when first presented with the issue. In reality, a RAG system would rely on maintaining a list of existing packages from repositories like PyPI or npm and checking AI output against that list in real time. But what if an attacker has already registered the hallucinated package? The model's output will appear correct, the package name will pass validation, and the installation will succeed, quietly importing malicious code.

To make matters worse, motivated attackers can go even further: they can build convincing README files, host fake documentation, and set up GitHub repositories that mimic legitimate projects. Even well-meaning developers who perform due diligence may be misled. In this way, package hallucinations represent not just a technical flaw, but a novel, AI-enabled escalation of traditional supply-chain attacks.

The Rise of "Vibe Coding"

A troubling development has been in the increase in popularity of "vibe coding," which is the practice of rapidly prototyping or building entire applications by following a model's suggestions without deeply understanding or reviewing the code. The focus is on speed and not security or verification. Developers with limited experience may accept AI-generated suggestions without hesitation, including hallucinated packages. What once required a strong foundation developed over much time and effort has become a single copy-paste operation. While this ease enables remarkable speed and creativity, it also significantly lowers the barrier for dangerous mistakes. Blindly accepting AI-generated instructions without verification invites exactly the kind of exploitation that package hallucinations make possible.

The Danger of Confidence

One underappreciated factor that makes hallucinations especially risky is the unshakable confidence with which LLMs deliver their outputs. These systems are optimized to sound fluent, authoritative, and helpful even when they're wrong. From a business and user experience perspective, this makes sense: users prefer decisiveness over hesitation. But in practice, confident errors are far more dangerous than hesitant ones. A model that fabricates a package name while speaking with assurance provides no cues for doubt. This sets up users, especially those unfamiliar with the underlying technology, for failure. Until models can be trained or configured to express appropriate levels of uncertainty, the burden of skepticism will remain entirely on the user.

Encouraging models to flag low-confidence outputs or provide explicit verifiability cues would help, but adoption of these techniques is still limited. For now, model confidence should not be mistaken for correctness. Developers and end-users alike must remain cautious, especially when AI outputs are actionable, such as installation commands, configuration suggestions, or dependency lists.

Trust, but Verify

This does not, in any way, mean abandoning AI copilots: they are wonderful. The lesson is shared responsibility. **Model architects** must treat hallucination risk as a first-class metric, expose confidence scores, and surface verifiability signals in the UI. **Developers** must layer controls: pin dependencies with hashes, run static analyzers in CI, and require human verification for any new package. Many companies host their own internal package repositories and hire companies to ensure that the packages hosted are legitimate and benign, a great practice but also only as good as the detection methods. **Organizations** must educate teams that “AI-generated” is not the same as “reviewed” and build a culture where speed never outranks security.

In short, AI is accelerating software creation at an unprecedented pace, but speed without scrutiny is an open invitation to attackers. Trust the assistant for inspiration and productivity; verify its suggestions before they ship.

> Like a self-driving car that navigates brilliantly 99% of the time but still requires the human driver to stay focused, LLMs are remarkable guides, provided we keep one hand on the wheel.

Conclusion: Charting a Safer Course

Returning to our original analogy, imagine if a GPS app that had guided you safely for years suddenly began inserting phantom destinations. Most drivers wouldn’t think to verify every route before departure. The same is true of developers using LLMs: when a system that you trust confidently hands you an answer, it’s human nature to follow it. But if that answer contains a hallucinated package name that has already been weaponized by an attacker, then even a simple "*pip install*" command becomes a liability.

AI has the power to elevate software engineering to new heights of speed and scale, but we must proceed with caution. As with any transformative technology, we must look beyond how well it works under ideal conditions and consider the risks it introduces at the margins. Package hallucinations are a clear warning: even when the model seems sure of itself, we must not be.

By embedding awareness, skepticism, and safety practices into our tooling and culture, we can embrace the power of AI without surrendering control to its most dangerous errors.

References:

[1] Berkay Kaplan and Jingyu Qian. A survey on common threats in npm and pypi registries. In MLHat, 2021.

[2] Shradha Neupane, Grant Holmes, Elizabeth Wyss, Drew Davidson, and Lorenzo De Carli. Beyond typosquatting: an in-depth look at package confusion. In USENIX, 2023.

[3] Jenny T Liang, Chenyang Yang, and Brad A Myers. A large-scale survey on the usability of ai programming assistants: Successes and challenges. In ICSE, 2024.

[4] Sonatype. 9th Annual State of the Software Supply Chain. https://www.sonatype.com/ state-of-the-software-supply-chain/ introduction, 2023.

[5] Fang Liu, Yang Liu, Lin Shi, Houkun Huang, Ruifeng Wang, Zhen Yang, and Li Zhang. Exploring and evaluating hallucinations in llm-powered code generation. arXiv:2404.00971, 2024.

[6] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. A survey on hallucination in large language

models: Principles, taxonomy, challenges, and open questions. arXiv preprint:2311.05232, 2023.

[7] Joseph Spracklen, Raveen Wijewickrama, A.H.M. Nazmus Sakib, Anindya Maiti, Bimal Viswanath, and Murtuza Jadliwala. We have a package for you! A comprehensive analysis of package hallucinations by code generating LLMs. In USENIX, 2025.

Article Categories: Security, Programming, AI/ML

Last updated July 3, 2025

Authors:

Joe Spracklen is a PhD student at the University of Texas at San Antonio studying hallucinations in code-generating LLMs. He is also an active duty Major in the United States Army most recently assigned to Army Cyber Command in a variety of roles, including Deputy Director of Assessments, Company Commander, and Mission Team Lead.

joseph.spracklen@my.utsa.edu

Dr. Raveen Wijewickrama is a postdoctoral researcher at the University of Texas at San Antonio. He is the manager and lead researcher of ScooterLab, an NSF funded micromobility testbed at UTSA that provides researchers with a rich data source to help improve the safety and efficiency of community infrastructure design. Dr. Wijewickrama received his PhD in 2024 from UTSA and also serves as the deputy director of SPriTELab, a cybersecurity lab focused on all aspects of cyber-physical systems and user privacy.

raveen.wijewickrama@utsa.edu

A.H.M. Nazmus Sakib is a PhD student at the University of Texas at San Antonio studying web fingerprinting and signal processing. He received his bachelor's degree from the University

of Dhaka in Bangladesh in Electrical Engineering and also works as a research assistant in ScooterLab at UTSA.

ahmnazmus.sakib@my.utsa.edu

Dr. Anindya Maiti is an Assistant Professor of Computer Science and Data Science and Analytics at the University of Oklahoma (OU), where he directs SecretLab. SecretLab's research focus lies in the areas of privacy and security, information assurance and network security, and applied AI/ML. He also serves as the Outreach Coordinator and Backend Systems Lead for ScooterLab, an NSF-funded micromobility test-bed at the University of Texas at San Antonio.

am@ou.edu

Dr. Bimal Viswanath is an Associate Professor of Computer Science at Virginia Tech whose research spans attacks using ML, attacks on ML systems, and the use of ML to enhance system defenses. He holds a Ph.D. from the Max Planck Institute for Software Systems and has received multiple honors, including the AI2000 Most Influential Scholar Award Honorable Mention for being among the top 100 most cited scholars in computer networking from 2009-2019.

vbimal@vt.edu

Dr. Murtuza Jadliwala is an Associate Professor at the University of Texas at San Antonio. He is the director of SPriTELab at UTSA and his research focuses on cybersecurity, cyber-physical systems, and user privacy. Dr. Jadliwala is the recipient of the UTSA Cloud Technology Endowed Fellowship and NSF Career Award, and his research has been funded through organizations like the Air Force

Research Lab, NASA, the Air Force Office of Scientific Research, and the National Science Foundation.

murtuza.jadliwala@utsa.edu

Log in to post comments

USENIX

Contact USENIX • Privacy Policy

© USENIX 2025
EIN 13-3055038
Website designed and built by Giant Rabbit LLC
Powered by Backdrop CMS